



EXECUTIVE TECHNICAL SUMMARY

MyInstantAI Agent Marketplace.

A condensed technical overview of the agent marketplace platform — scale, architecture, the catalogue, runtime, security, and the path to production — companion to the full 23-section due-diligence pack.

AUDIENCE

MyInstantAI engineering & technical due diligence

SNAPSHOT

commit `fb97438` · branch `main` · 2026-08-25 · v0.1.0

STACK

Next.js 15 (App Router) · React 19 · TypeScript 5 · Node 20 ·
pnpm workspaces · Postgres (pg, no ORM) · Docker → Railway
(Azure Container Apps = migration target)

REPOSITORY

`github.com/MalcolmGov/miai-agent-marketplace` (private)

SCALE

406,319 authored lines · 1,212 files · 500-agent catalogue
(~299,508 lines of catalogue IP) · ~75,900 lines platform
runtime · 69 API routes · 8,526 evals · 2,128 tools

How to read this summary

This is the ~8-page executive brief for the platform. Every figure is measured from source at the snapshot above. The companion **151-page Due-Diligence Pack** (23 sections, 00–22) proves each claim in depth; this summary is the map. It is deliberately technical — no pricing or commercial terms appear here.

1 What it is

The **MyInstantAI Agent Marketplace** is a running, multi-tenant **Agents Marketplace + Agent Runtime**: a first-party surface where a partner's customers browse a catalogue of AI agents, rent one, configure it (persona, knowledge, connected Actions, branding), and run it — metered against a prepaid token wallet — across web, an embeddable widget, Telegram, WhatsApp, a native-app WebView, and a machine-readable MCP interface. It serves two audiences from one codebase: **business agents** (rented per workspace) and **consumer specialists** (a personal-assistant line with durable memory).

It is **not a prototype behind a flag**. The complete product loop — *browse → rent → configure → connect Actions → paste knowledge → sandbox/live chat → embed → prepaid-token metering* — runs today on mockable rails, backed by the full 500-SKU catalogue, an 8,526-case eval harness, Postgres persistence, five delivery channels, an OAuth connector framework, and a fail-closed token wallet. What stands between staging and production is **wiring, not invention**: three mock adapters (identity, wallet, model) swap for MyInstantAI's live endpoints via configuration, and one hosting topology is chosen. The adapter seams, production guards, health endpoint, and migration runbook already exist to make that swap low-risk.

At a glance — measured at `fb97438`

DIMENSION	VALUE	DIMENSION	VALUE
Total authored	406,319 lines	Agent catalogue (SKUs)	500 (100 families × 5 markets)
Files	1,212	Catalogue files on disk	552 <code>*.agent.json</code> (+52 aliases)
Catalogue IP (74% of repo)	299,508 lines	Consumer specialists	17
Platform runtime (hand-written)	~75,900 lines	Eval cases (catalogue)	8,526 (~15/agent)
API routes / pages	69 / 39	Tool definitions (on-disk)	2,128
SQL migrations / ADRs	5 / 5	Delivery channels	5 (web, embed, Telegram, WhatsApp, MCP + App)

Scale note. The headline is **406,319 lines**, of which **299,508 (74%)** is the catalogue — the 500 agents' `system_prompt` + `knowledge` + `tools` + `guardrails` + `evals`. That catalogue *is* the core asset; quoting only the hand-written runtime understates the platform roughly four-fold.

2 Architecture at a glance

The system is a pnpm-workspace monorepo with a clean layering: a **Next.js 15 App Router** front end and API layer (`apps/web`), a set of shared server libraries (~85 modules), and five packages — the **agent-protocol** (shared contracts), the **runtime engine**, the **connectors**, the generated **presets**, and the **wallet-adapter**.

A single chat turn flows: **browser/channel** → **Next.js API route** (auth → RBAC → zod validation) → **turn orchestrator** → `runTurn()` in the **runtime engine** → **model adapter + connector/tool calls under guardrails** → **SSE stream back** → **memory persisted**. The engine is generic; all vertical knowledge lives in the catalogue.

The design is **adapter-first**, which is what makes the partner cutover a configuration change rather than a rewrite:

SEAM	MOCK (STAGING TODAY)	LIVE (PRODUCTION)	SWITCHED BY
Identity	shared demo / mock	OIDC Bearer (B2B), Google OIDC (consumer)	<code>MIAI_AUTH_MODE</code> + issuer/JWKS
Wallet	in-memory mock	partner debit/balance/top-up API	<code>MIAI_WALLET_MODE</code> + wallet URL/key
Model	deterministic mock	OpenAI / Anthropic / Azure / gateway	<code>MIAI_MODEL_MODE</code> + keys
Connectors	stubbed under <code>SANDBOX_MODE</code>	live OAuth token execution	provider credentials

Every tenant is isolated by a `tenant_id` predicate in **every** query — its own public key, knowledge base, document index, conversation history, action configuration, usage ledger, and domain allowlist. Cross-tenant reads are structurally impossible.

3 The catalogue – the core asset

The catalogue is **500 SKUs = 100 agent families × 5 regional market packs** (US, EU, Africa incl. ZA, Asia, Oceania – exactly 100 each; ADR 0001), plus **17 consumer specialists**. On disk that is 552 `*.agent.json` files (500 canonical + 52 legacy deep-link aliases) totalling **299,508 lines** of authored IP.

Every agent is a **real runtime, not a prompt wrapper** – a flat JSON bundle carrying five fields:

FIELD	WHAT IT ENCODES
<code>system_prompt</code>	the persona and operating rules
<code>knowledge</code>	grounding content (auto-switches from prompt-stuffing to vector retrieval above ~large knowledge bases; docs to 200K chars, 20/agent)
<code>tools</code>	the Actions the agent may call – 2,128 tool definitions across the catalogue
<code>guardrails</code>	per-persona refusals enforced in the runtime (e.g. an IT agent never touches a password/OTP; an insurance agent never promises a payout)
<code>evals</code>	behavioural test cases – 8,526 across the catalogue (~15/agent), a regression harness for grounded answers, confirm-before-write, and cross-tenant refusal

Agents are tiered (Standard / Pro / Enterprise); the tier model is encoded as constants in `agent-protocol`. Pro-tier agents run a bounded **plan → act → observe** loop (≤ 3 steps, credit-checked between each). Every action call and plan is logged and attributable to the owner – a glass-box runtime.

4 Runtime & AI platform

The heart is `packages/runtime` (`runTurn()`, `createModelAdapter()`): a provider-agnostic engine supporting **five model modes** (mock / OpenAI / Anthropic / Azure OpenAI / gateway), guardrail enforcement, hybrid semantic retrieval (lexical + vector RAG), 20+ per-vertical workflow modules, and the bounded reasoning loop. Critically, **metering fails closed**: before any model call the runtime compares lifetime usage against credits and, at zero, returns a top-up message and makes **no provider call and spends nothing** — so a merchant can never run up an unfunded bill, and there is no surprise wholesale line.

5 Integrations & channels

A connector framework ([packages/connectors](#)) provides **OAuth 2.0 + PKCE** onboarding with AES-GCM-sealed token storage and live execution (Slack, webhook, and MCP execute today; the broader provider set – Google, Microsoft M365, Teams, Shopify, HubSpot, Xero, QuickBooks, Calendly, Zendesk and more – acquire tokens in staging). Agents reach customers over **five channels**: web chat, a 6-KB zero-dependency **embeddable widget** (one script tag, derives its own origin so it serves from the partner's domain unchanged), a **Telegram** webhook, **WhatsApp**, a native-app **WebView**, and an **MCP server** (JSON-RPC, six tools) that lets a partner's own assistant rent and deploy agents conversationally. Outbound webhooks are HMAC-SHA256 signed.

6 Security, privacy & compliance

AREA	WHAT IS IN PLACE
Identity	B2B OIDC Bearer (issuer/audience/JWKS, <code>workspace_id</code>); consumer Google OIDC (Auth-Code + PKCE, HS256-pinned HttpOnly session, open-redirect guard); signed embed keys; webhook HMAC
Isolation	<code>tenant_id</code> predicate on every read/write; owned agents cannot be re-claimed; payment references are tenant-scoped (404, not a credit); competitors can coexist as two blind tenants
Hardening	dual-flag production guards, CSP + embed CORS + <code>frame-ancestors</code> allowlist, DNS-pinned SSRF check on outbound URLs, AES-GCM-sealed OAuth tokens, agent-IP redaction (prompts/guardrails/evals stripped from public reads)
Prompt-injection	only the model's own output is parsed for tool calls; visitor text is scrubbed before entering the transcript; tool results re-enter as data; MCP tool descriptions sanitised; transcript turns newline-collapsed
Privacy / regulatory	DSAR export + erase, consent, audit log, retention, PII redaction, AI disclosure; regional market packs encode US CCPA/TCPA, EU GDPR, Africa POPIA (incl. ZA), Asia PDPA, Oceania AU/NZ — per-pack, not one global prompt; a Trust Center with honest Live / Partial / Planned tags

7 Data & operations

Persistence is **Postgres** (`DATABASE_URL`) via `pg` (no ORM) with a JSON file-store fallback for staging; five ordered SQL migrations (`001_init` → `005_consumer_reminders` ; ADR 0003). Consumer memory spans session, durable facts, a goals/people life-graph, a per-agent knowledge base, and reminders — keyed to `(tenant, person)`. The platform ships as a multi-stage **Docker** image to **Railway** today (health at `/api/health`, deep config introspection, 503 on degraded), with **Azure Container Apps** (Azure OpenAI + App Insights, Bicep IaC) as the stated migration target. Quality is gated by **5 CI workflows** (required `quality` check: build → typecheck → tests → catalogue integrity → static eval), **unit + Playwright e2e** suites, and the **8,526-case eval harness** run nightly.

8 Path to production – cutover

The runtime already derives its host at request time, so several routes to production exist; pick by constraint, not maturity:

TOPOLOGY	WHAT IT IS	PARTNER EFFORT	TIME
A – API + partner storefront	partner calls the REST/MCP API from their own UI; Zara hosts	High (build UI)	Available now
B – partner domain, hosted (<i>recommended launch</i>)	partner domain resolves to the hosted runtime; partner brand/TLS/URLs	DNS + config only	~2 weeks
C – embedded in partner console	marketplace + setup render inside the partner app via signed SSO handoff, no second login	Low (embed + SSO)	~3 weeks
D – partner Azure tenancy	runtime ships as a container into the partner's subscription; partner owns the data plane	Medium	Azure-tenancy option; gated on managed-Postgres migration first

14-day technical fast-track: Days 1–3 OIDC login end-to-end · 4–7 wallet debit + model gateway on one hero agent (activate → chat → bill) · 8–11 custom domain + OAuth redirects + UAT on a go-live shortlist · 12–14 cutover rehearsal, mock rails off, Live Ops green on the partner hostname.

Required from MyInstantAI to go live: production OIDC (issuer, audience, JWKS, sample JWT) · production wallet API (debit / balance / top-up) · model gateway (base URL, auth, tool-calling) · cloud subscription + region + production hostname/DNS · WhatsApp WABA/BSP.

Readiness snapshot

Live: marketplace + Agent Studio loop · 500-SKU catalogue + 8,526 evals · runtime engine + guardrails + RAG + fail-closed metering · health endpoint · five channels · sandbox chat on mock model · OAuth token acquisition. **Live code, needs partner credentials:** OIDC auth, wallet debit, live model providers. **Partial:** live *execution* handlers beyond Slack/webhook/MCP. **Planned/optional:** topology-D Azure-tenancy deployment (after the managed-Postgres migration).



9

The full due-diligence pack

This summary condenses a **23-section, 151-page technical pack** (companion document), generated from a direct read of source and independently scale-verified. Its sections: 00 Handover Readiness · 01 Executive Summary · 02 Codebase Statistics · 03 Project Structure · 04 Frontend · 05 Backend · 06 AI Platform · 07 API Surface · 08 Data Architecture · 09 Integrations · 10 Security · 11 Compliance · 12 DevOps & Cloud · 13 Dependencies · 14 Performance & Scalability · 15 Testing & Evals · 16 Documentation · 17 Engineering Hygiene · 18 Design Patterns · 19 Marketplace Metrics · 20 Complexity & Rebuild Effort · 21 Visual Reports · 22 Handover & Cutover Plan — plus appendices (complete endpoint index, environment-variable catalog, dependency manifest, documentation index) and an open-items ledger.